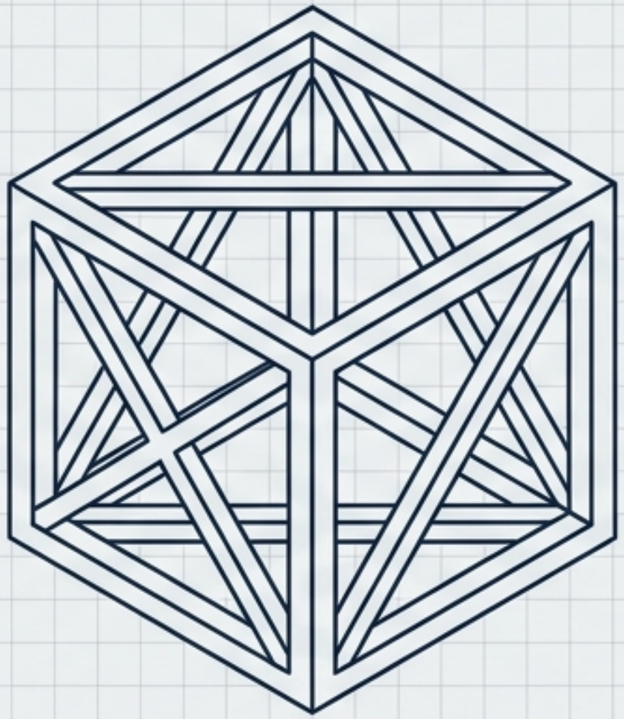


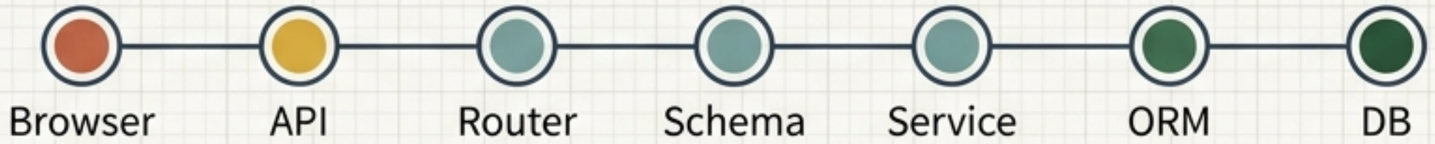


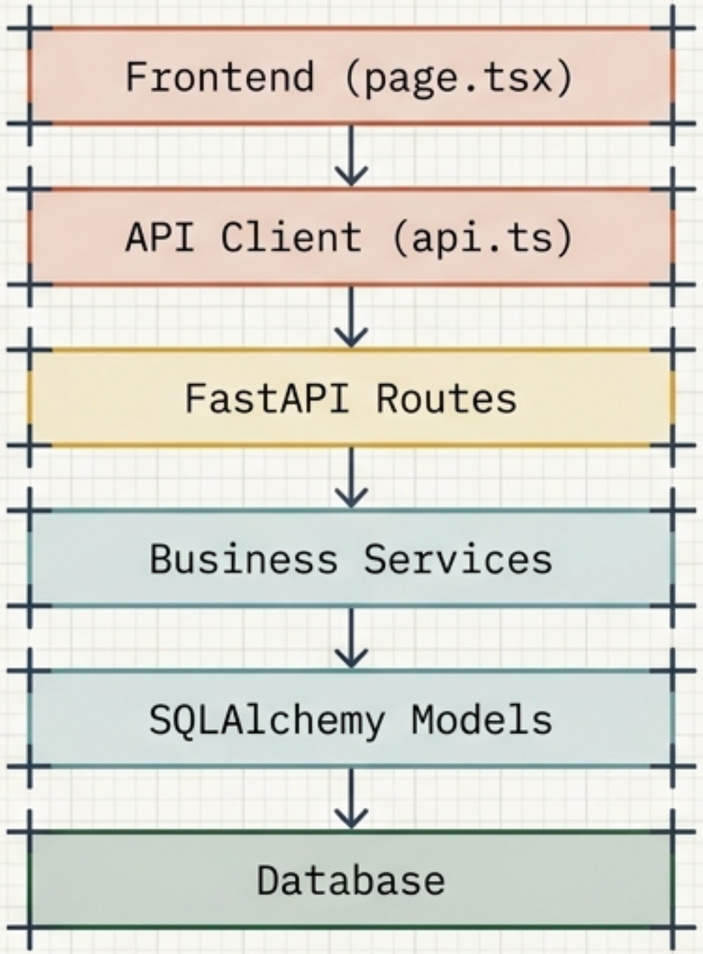
Tracing a Request Through the Stark HQ Architecture

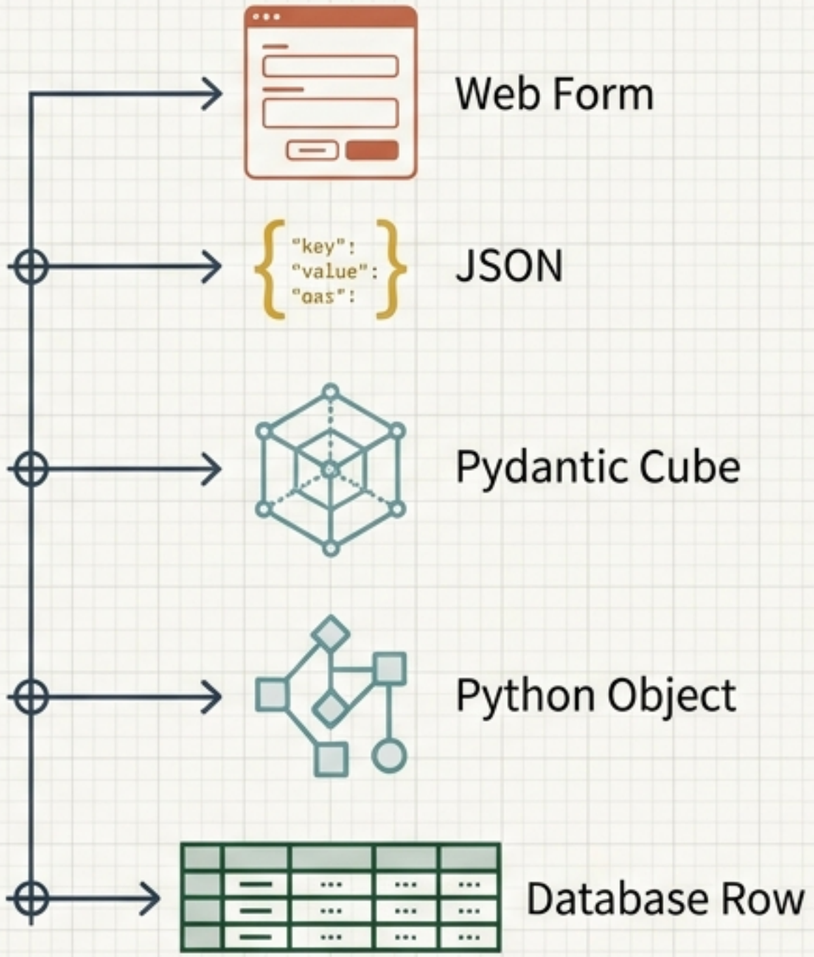
Understanding 40+ files by following a single feeding record from the React UI to the PostgreSQL database.

Gemini Notebook

The Spine of the System



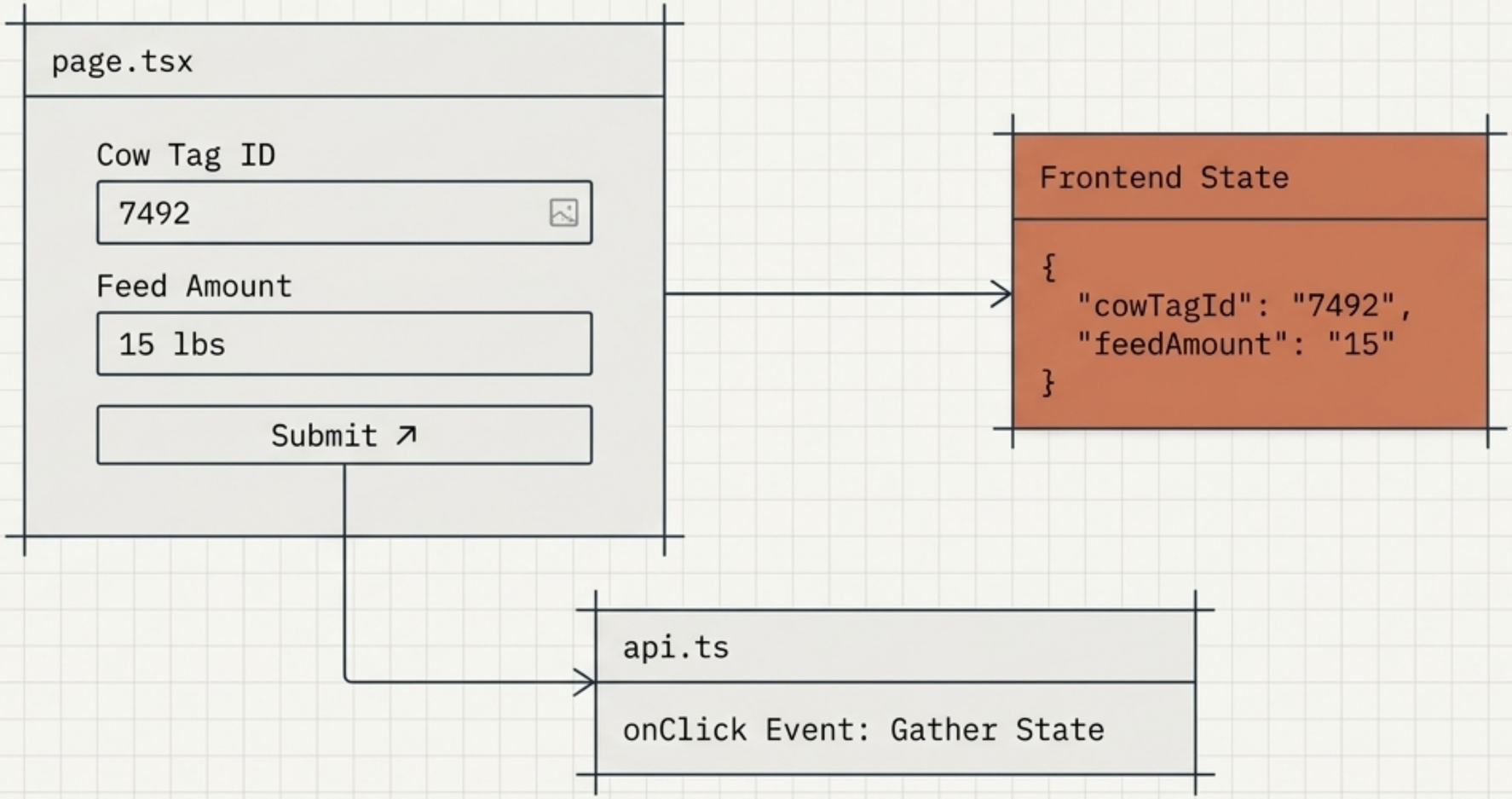




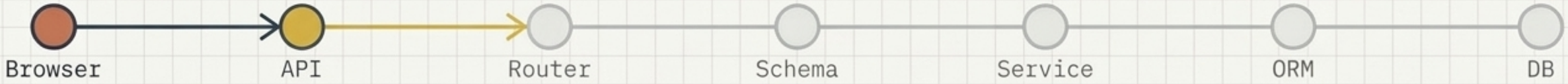
The backend is not a collection of random files. It is an assembly line designed to transform, validate, and store state.

Gemini Notebook

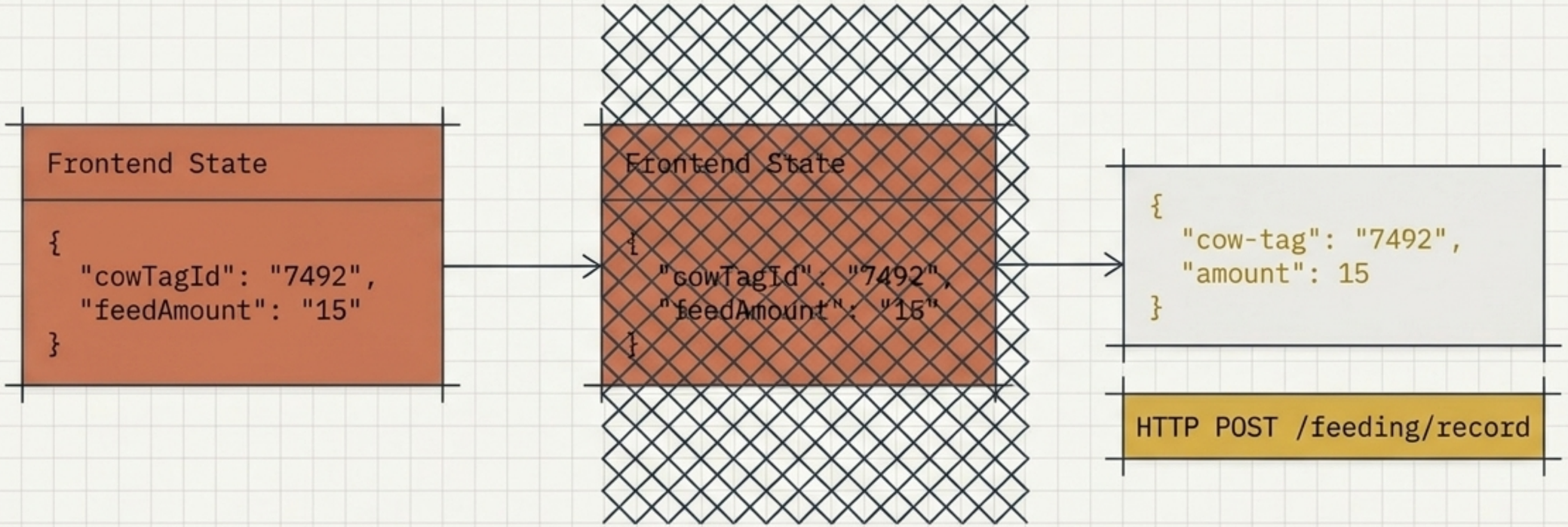
The Trigger: Capturing Frontend State



Gemini Notebook



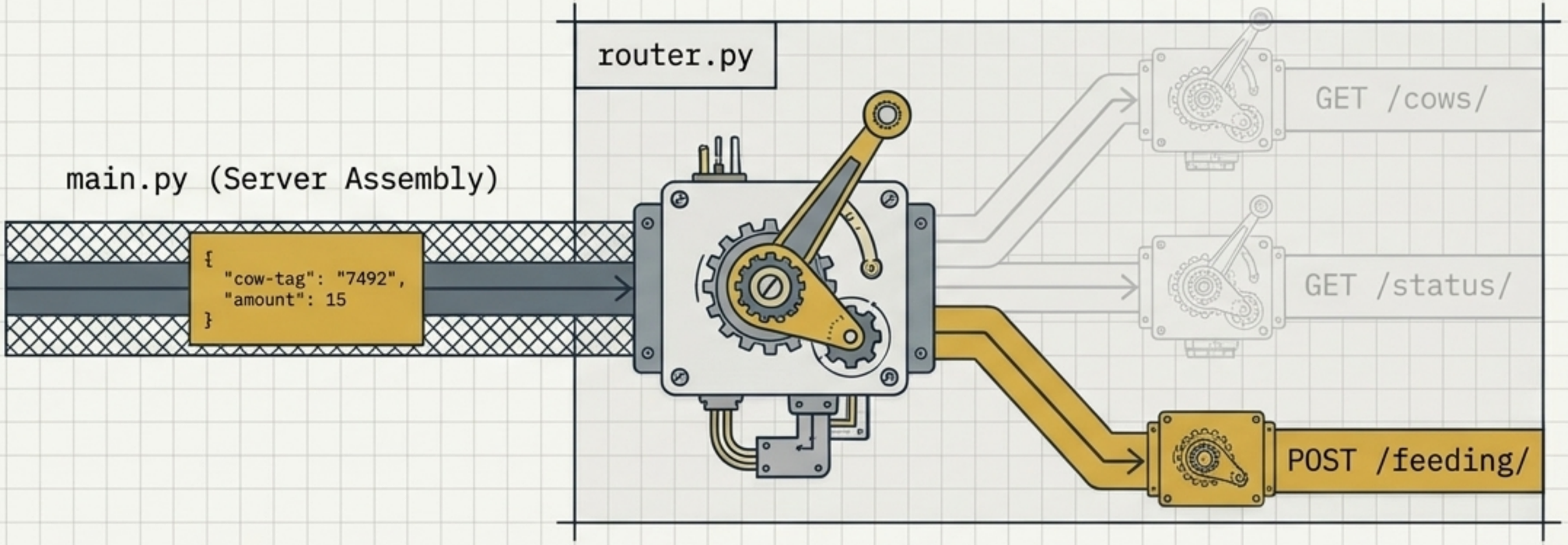
Crossing the Boundary via HTTP



Data is entirely **untrusted** and **structurally unverified** at this stage.

Gemini Notebook

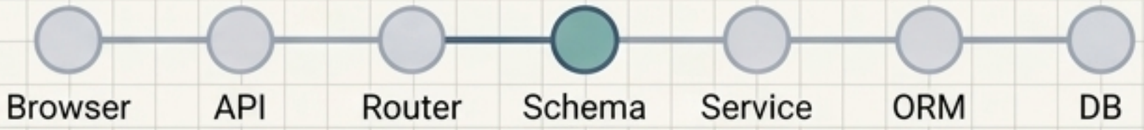
Traffic Control and the FastAPI Router



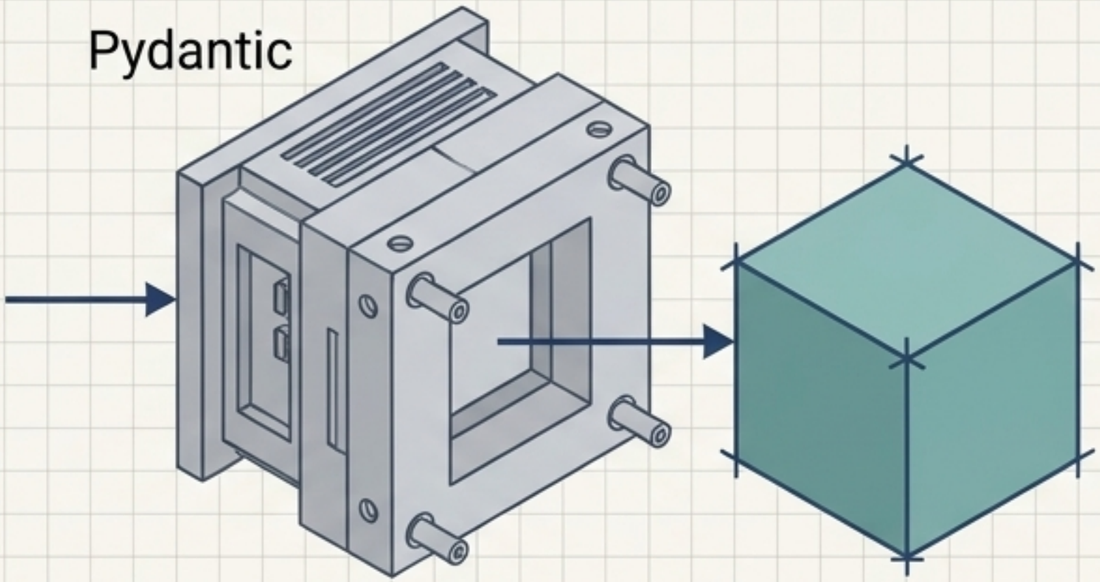
Routers contain **zero business logic**. Their sole job is receiving the request and pointing it to the **correct endpoint.**

Gemini Notebook

Gate 1: Structural Validation




```
{
  "cow-tag": "7492",
  "amount": 15
}
```



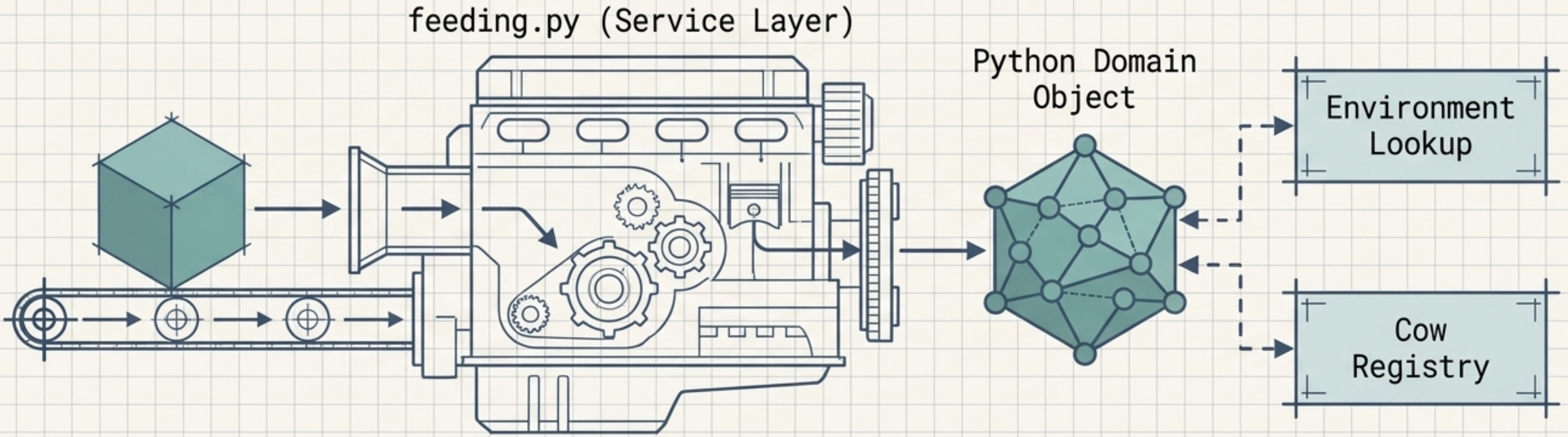


The schema answers ‘Does this data have the right shape?’ before the application ever considers farming logic.

Gemini Notebook

The Brains of the Operation



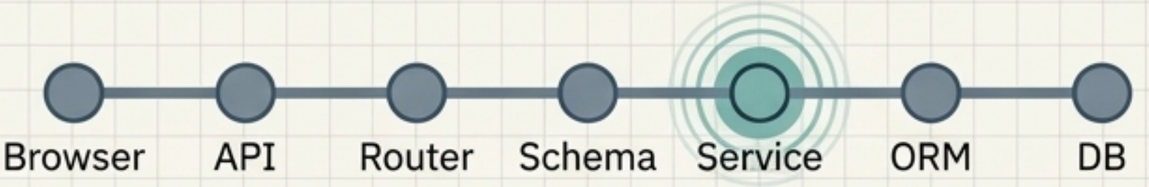


Why isn't this logic inside the API endpoint?

Because services decide what should happen independently of how the data arrived (HTTP, CLI, or internal cron job).

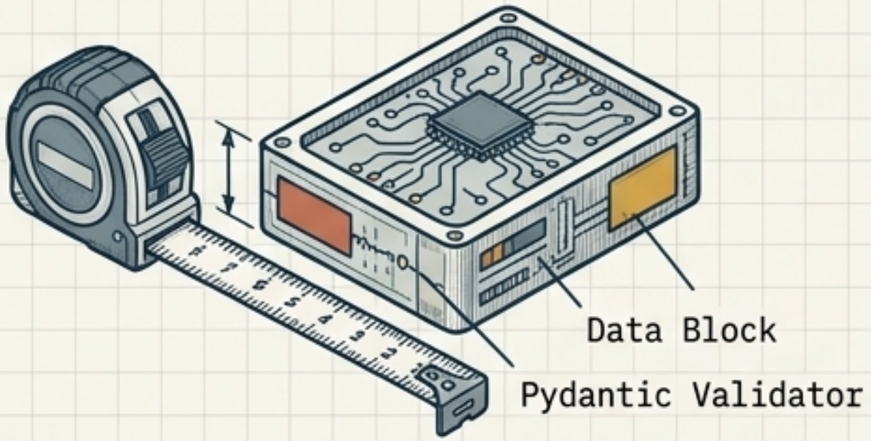
Gemini Notebook

Technical Validation vs. Business Rules



Gate 1: Schema

Technical Validation



Is amount an integer?

Handled by Pydantic

Gate 2: Domain

The Laws of the Farm



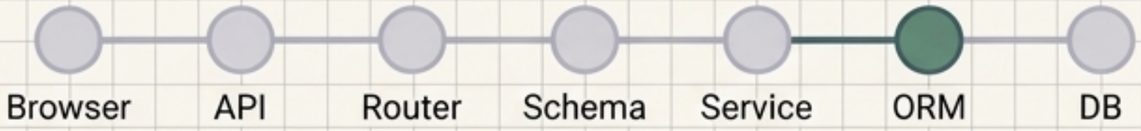
Can this specific cow safely consume 15 lbs of this feed right now?

Handled by Business Logic

Passing Gate 1 means the data is readable. Passing Gate 2 means the action is legal.

Gemini Notebook

The Shape-Shifter: Python to SQLAlchemy



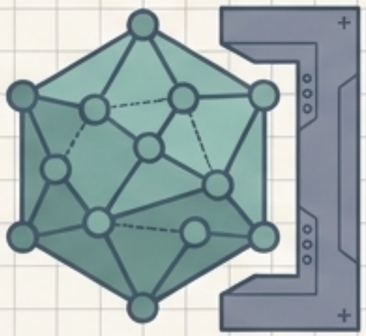

```python
class FeedingRecord(Base):
    __tablename__ = 'feeding_record'

    id = Column(Integer, primary_key=True)
    cow_id = Column(Integer, ForeignKey('cow.id'))
    feeding_date = Column(DateTime)
    quantity = Column(Float)
    cost = Column(Decimal)
    feed_type_id = Column(Integer,
        ForeignKey('feed_type.id'))

    cow = relationship("Cow",
       back_populates="feeding_records")
    feed_type = relationship("FeedType",
       back_populates="feeding_records")
```

FeedingRecord	
cow_id	int
id	int pk
cow_id	int fk
feeding_date	date
quantity	decimal
cost	decimal
feed_type_id	text

Cow	
supplier_id	int
state_id	sRt
sink_id	sRt
cow_id	int fk
coures_lop_names	varchar
tree	varchar
larts_side	data
acquisition_splot	varchar
innolation_date	data
purclısition_price	decisary
debut	varchar
warehouse_id	int



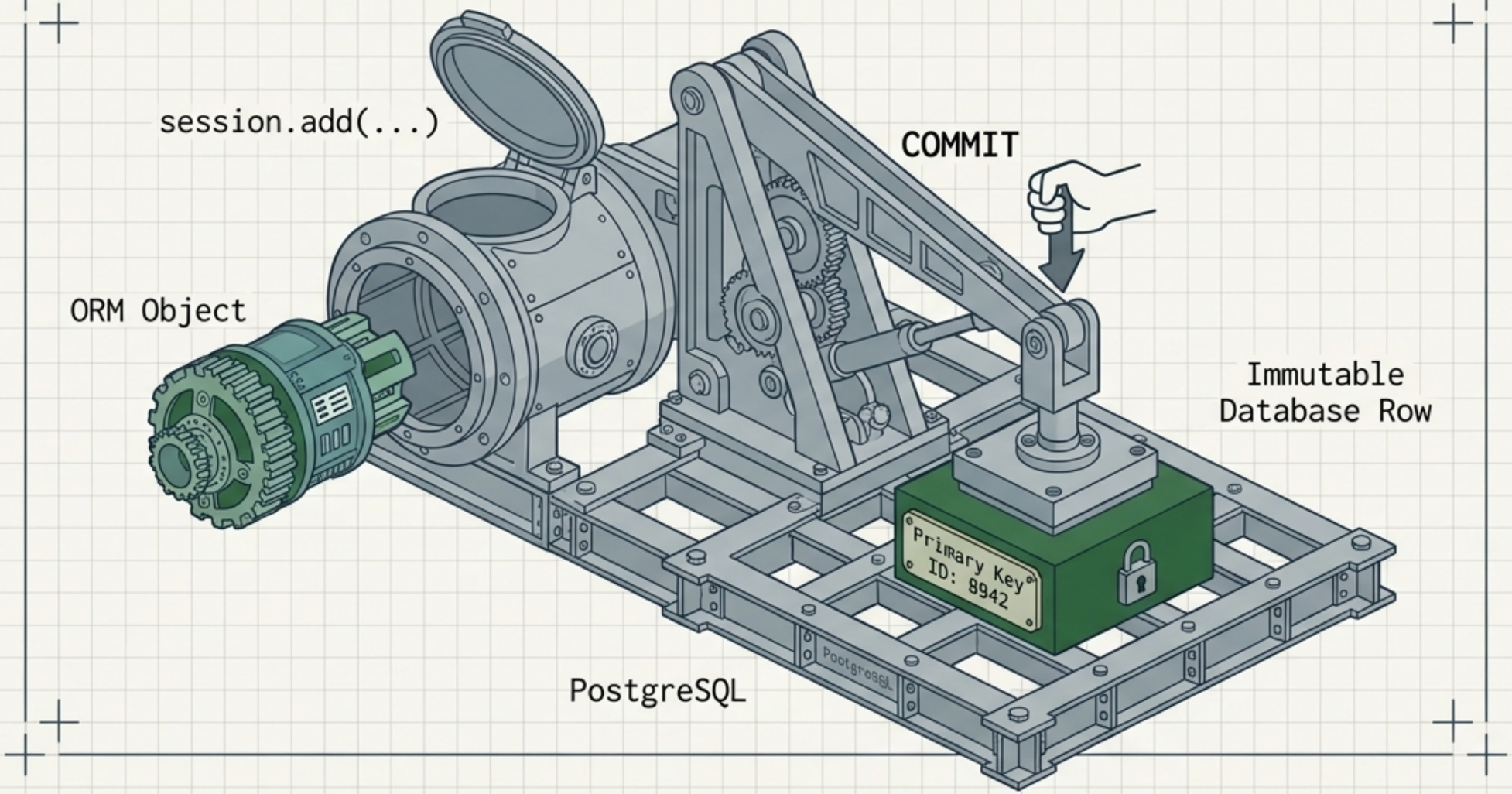
ORM Model Form

Database schema expressed natively in Python.

Gemini Notebook

The Anchor: Committing to the Database

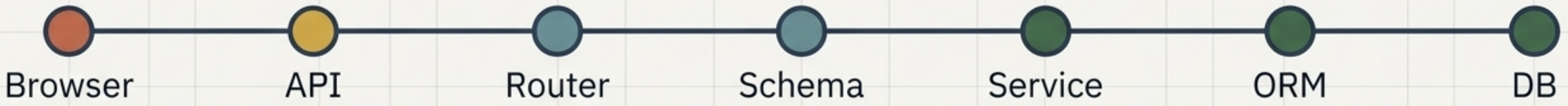
Browser · API · Router · Schema · Service · ORM · **DB**



Only at this exact moment does the abstract operation become permanent reality on the farm.

Gemini Notebook

Synthesis: The ‘Why’ of Separation



Layer	Core Responsibility	File Context
Router	Receives request	router.py
Schema	Validates shape	Pydantic models
Service	Decides what happens	feeding.py
Model	Represents persistent data	herd.py / feeding.py
Database	Stores data permanently	database.py

Bypassing this separation creates brittle, unscalable CRUD scripts. Adhering to it creates a robust, predictable machine.

Gemini Notebook



Testing the Machine: Where Does It Break?

Scenario A: User sends string 'five' instead of number 5.

Schema Layer (422 Unprocessable Entity). Never reaches database.

Scenario B: User tries to feed a cow that was sold yesterday.

Service / Domain Layer (Business Rule Violation).

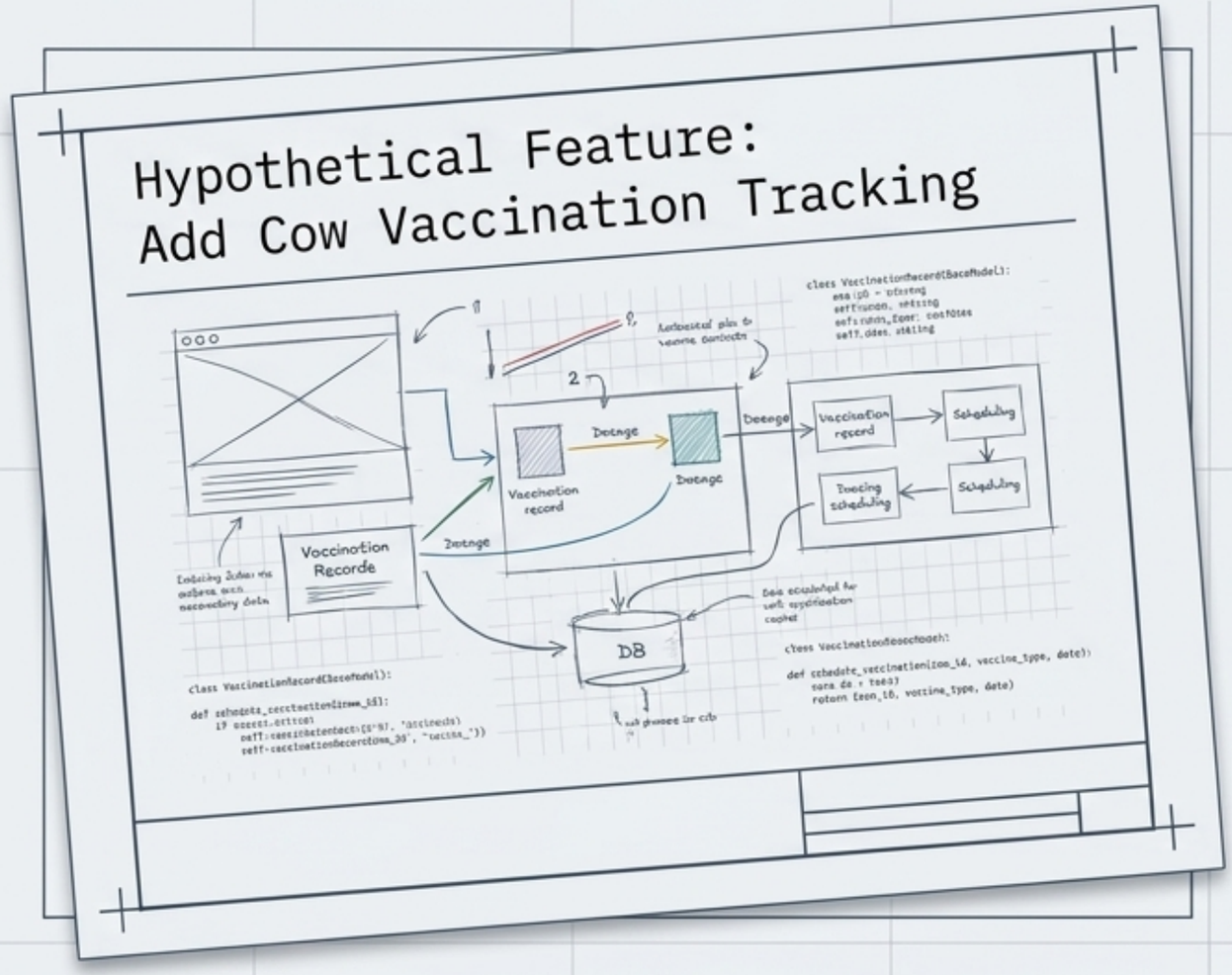
Scenario C: Concurrent process deletes cow milliseconds before commit.

Database Layer (Foreign Key Constraint failure during session.commit).

Gemini Notebook

The Architect's View

Browser API Router Schema Service ORM DB



New Table?
Yes, mapped in health.py ✓

New Pydantic Schema?
Yes, structural validation for dosage ✓

New Service?
Yes, business rules for vaccination intervals ✓

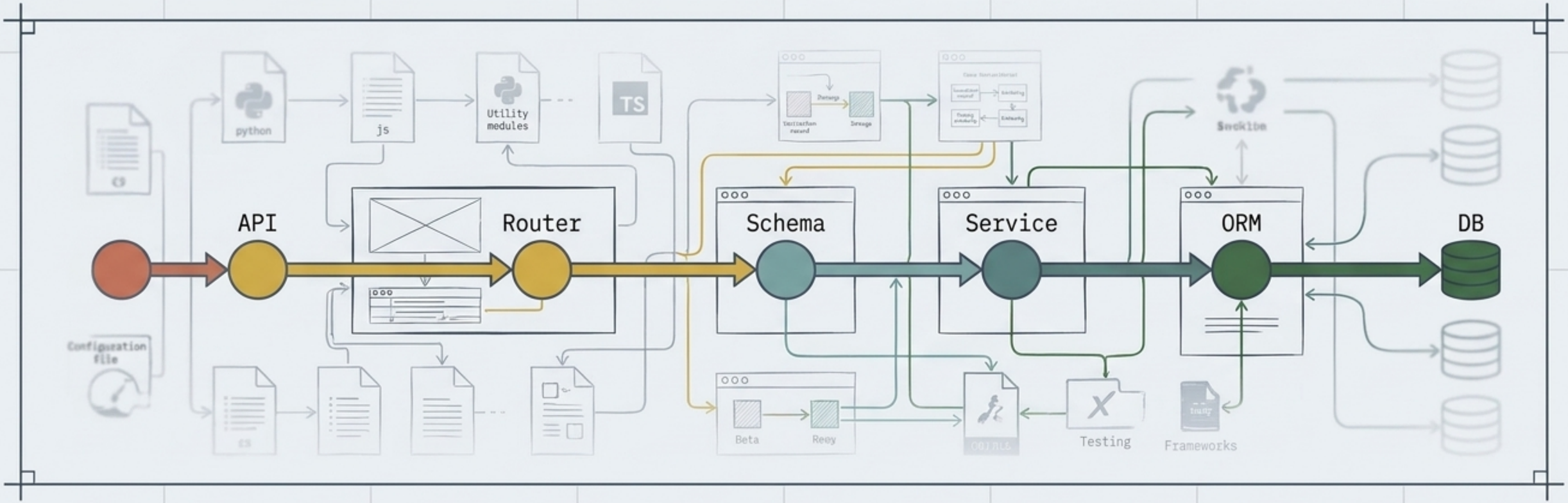
New Endpoint?
Yes, mapped in router.py ✓

The architecture removes the guesswork. You design the system; the framework dictates where every piece belongs.

Gemini Notebook

You Now Know the Machine





Data flows. **State transforms.** **Dependencies resolve.**

The codebase is no longer a collection of 40 isolated files. It is one cohesive, predictable assembly line. Go build.

Gemini Notebook